本地知识助手 Agentic RAG产品设计文档

苑夫唯

2026-04-19

本地知识助手 Agentic RAG产品设计文档

Local AI system for reliable knowledge retrieval with source citation

本项目是一个面向个人知识管理场景的本地 AI 检索系统。它通过本地 embedding、向量数据库、来源引用和轻量 GUI，把 Obsidian 笔记与本地文档转化为可检索、可追溯、可迁移的个人知识层。

GUI 总览

1. 项目概览

这个项目不是单纯的技术 demo，而是一个完整的 AI 产品案例：

- **产品侧**：定义个人知识检索痛点、目标用户、核心场景和成功指标。
- **系统侧**：搭建本地文档索引、向量检索、LLM 回答和引用溯源流程。
- **体验侧**：提供 Web GUI，让日常使用不依赖终端。
- **验证侧**：通过真实问题集检查系统是否能找回资料、引用来源并辅助复盘。
- **产品化侧**：保留从个人工具扩展到小团队知识工作台的路径思考，但不把当前版本包装成已商业化产品。

当前版本已完成 Obsidian Markdown 文件批量索引、本地 embedding、Chroma 向量检索、带引用回答、本地 GUI 和换电脑迁移说明。

2. 问题定义

2.1 关键词搜索不足

个人知识库长期积累后，资料会分散在 Obsidian、本地文档、项目资料和面试素材中。传统搜索适合“知道关键词”的场景，但不适合“记得大概意思、忘了具体文件”的场景。

典型问题包括：

- 文件名和关键词不一定记得。
- 同一个概念在不同笔记里可能有不同表达。

- 长文档中的局部信息很难被快速找回。
- 项目复盘、面试准备、写作时经常需要跨文件找依据。

2.2 直接问 LLM 容易产生幻觉

如果不先检索用户自己的资料，直接让 LLM 回答，常见风险是：

- 回答流畅但没有依据。
- 混入通用知识，无法匹配用户自己的笔记。
- 对项目数据、文件状态和迁移规则产生错误推断。
- 面试准备场景下容易把未验证的说法当成真实经历。

因此，本项目把“来源引用”作为核心能力，而不是附加功能。

2.3 本地隐私是硬约束

个人知识库中包含求职资料、项目复盘、私人笔记和本地路径，不适合整体上传到云端向量库或在线知识库服务。本项目选择 local-first 路线：

- 文档保留在本地。
- embedding 在本地生成。
- 向量库在本地保存。
- GUI 在本机运行。
- 公开展示只使用脱敏截图和样例。

---

3. 目标用户

用户类型	典型特征	主要需求
知识工作者	有大量笔记、资料和项目沉淀	快速找回历史资料和决策依据
求职转型者	需要整理作品集、项目经历和项目复盘	从分散笔记中提取可讲述内容
AI 产品学习者	想把 AI 工具接入个人工作流	理解 RAG 如何服务真实场景

当前版本首先服务个人使用场景，再通过公开作品集方式展示产品思考、系统设计和可运行成果。

---

4. 核心使用场景

场景	用户问题	产品价值
找回历史规则	“我之前写过的 AI 协作规则在哪里？”	用语义检索找回相关笔记
面试项目准备	“本地知识助手 Agentic RAG 项目如何介绍？”	聚合项目简介、压缩版和验证资料
索引维护	“文件改动后怎么办？”	找到索引更新和迁移说明
跨项目复盘	“本地知识助手 与企业知识库 RAG 有什么区别？”	从多个项目资料中召回依据
换电脑迁移	“换电脑时需要迁移哪些内容？”	明确代码、模型、向量库、缓存和路径配置

---

5. 产品目标与非目标

产品目标

- 支持 Obsidian Markdown 和本地文本类文档索引。
- 支持自然语言语义检索。
- 支持基于来源的回答生成。
- 回答必须展示引用来源。
- 支持低配置电脑本地运行。
- 提供 GUI，降低日常使用门槛。
- 形成可公开展示的作品集案例。

非目标

- 不做企业级多用户权限系统。
- 不默认上传私人资料到云端。
- 不追求大模型复杂推理。
- 不把生成答案当成最终事实。

- 不在当前阶段覆盖所有扫描 PDF、图片和音视频。
- 不替代 Obsidian 的写作和链接能力。

6. 用户流程

用户查询流程

日常使用流程：

1. 用户把资料写入 Obsidian 或放入指定本地文档目录。
2. 用户在 GUI 中执行索引任务。
3. 系统读取文件、转换文本、切分 chunks、生成 embedding。
4. 用户输入问题。
5. Agent 判断任务类型：直接检索、来源问答、索引维护说明，或需要用户补充条件。
6. 系统召回相关片段，并按来源相关性组织候选内容。
7. 系统基于召回内容生成回答。
8. GUI 展示回答、来源文件和命中片段。
9. 用户根据引用回到原文复核，必要时把高价值回答沉淀回 Obsidian。

检索工作台

7. GUI 设计

GUI 的目标是把本地知识助手 从“终端工具”变成“日常可用的知识检索工作台”。页面不强调技术炫技，而强调快速检索、来源可信、状态可见和操作清楚。

模块	作用	关键设计
检索工作台	先找到相关资料	搜索框高优先级、结果卡片可快速扫读
来源问答	基于资料组织回答	回答与引用同时出现，来源优先
索引任务	管理资料进入系统	展示文件数、chunk 数、执行状态
状态面板	降低系统黑盒感	展示模型、索引规模和运行状态

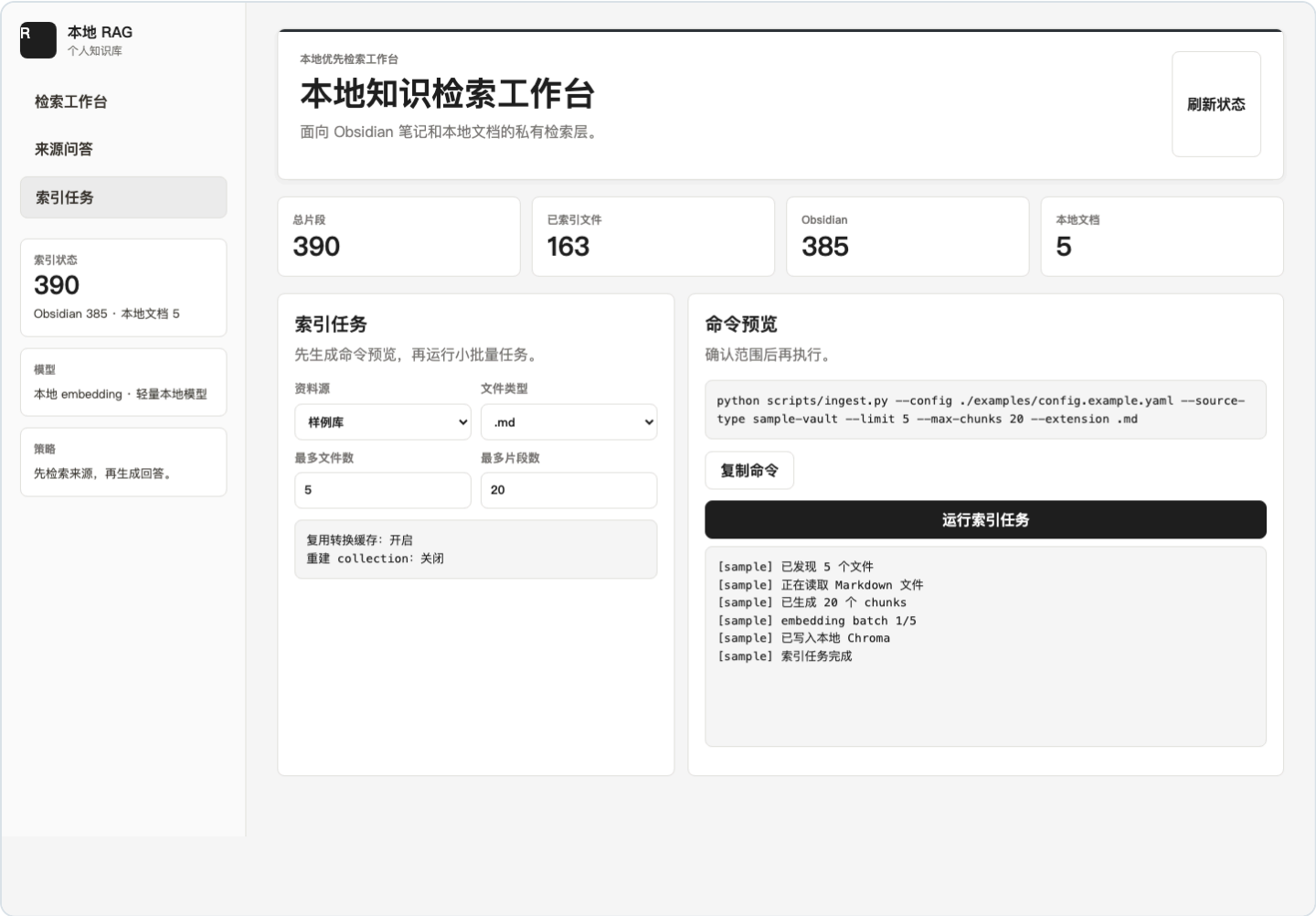
来源问答

来源问答页面用于“先找资料，再生成答案”。它要求回答和引用同时出现。如果检索依据不足，系统应提示不确定，而不是编造答案。

带引用回答

索引任务

索引任务页面用于触发和观察索引过程。考虑到本机硬件是普通个人电脑，系统支持小批量索引，并提示大批量索引可能占用 CPU。



索引任务界面

8. 系统架构

系统架构

层级	职责	设计重点
数据源层	Obsidian Markdown、本地文档	保持原始资料不被修改
处理层	转换、清洗、切分	让长文档变成可检索片段
向量层	embedding、Chroma	支持语义召回
检索层	query embedding、相似度搜索	优先找准来源
生成层	prompt、LLM 回答	基于来源回答，不脱离上下文
体验层	Web GUI	让非终端使用成为可能

9. 关键设计决策

9.1 为什么用 RAG，而不是微调

本项目的资料会频繁变化，包括项目文档、面试材料、规则说明和迁移记录。微调无法有效解决“最新资料在哪里”和“答案依据是什么”的问题。

RAG 更适合这个场景，因为它可以：

- 动态接入新增文档。
- 回答时引用具体来源。
- 在资料变化后重新索引。
- 避免把个人资料固化进模型权重。

9.2 为什么本地部署，而不是云端知识库

本地部署的核心原因是隐私和控制权。个人知识库不是公共资料，其中可能包含真实路径、求职准备、项目复盘和私人笔记。

代价是模型能力受本机硬件限制，索引速度不如云端服务，跨设备使用需要迁移说明。但对于个人知识库 MVP，本地优先更符合真实约束。

9.3 为什么引用来源是核心功能

如果没有引用来源，RAG 系统很容易变成“看起来像知道”的聊天机器人。个人知识库场景里，用户最需要的是回到原文复核。

本项目的产品原则是：**先展示来源，再组织答案**。

9.4 为什么需要 GUI

命令行可以完成索引和检索，但不适合长期日常使用。GUI 的价值是降低使用门槛，让检索、问答、索引和状态可见，也让作品集展示更像产品，而不是脚本集合。

9.5 为什么 rerank 不是当前 P0

RAG + rerank 是合理方向，但当前版本优先做 lightweight MVP。小规模个人知识库下，基础向量检索已经能覆盖主要场景；rerank 会增加模型调用、延迟和本地资源占用。因此 rerank 被放入后续路线。

10. 数据与索引机制

10.1 数据接入

当前优先接入：

- Obsidian Markdown 笔记。
- 本地文本类文档。
- 少量文档转换 smoke test 样例。

暂不默认接入：

- 证件、合同、财务、医疗等敏感文档。
- 未下载到本地的云端占位文件。
- 扫描 PDF 和图片中的 OCR 内容。

10.2 Chunk 策略

文档切分的目标不是越细越好，而是让每个片段同时满足：

- 信息足够完整。
- embedding 语义清晰。
- 检索结果便于阅读。
- 能回到来源文件定位。

过大的 chunk 会导致召回不精确；过小的 chunk 会丢失上下文。因此系统采用偏稳妥的结构化切分，并保留来源元数据。

10.3 Embedding 与向量库

选择本地 embedding 的原因是私人资料不出本机、成本可控、适合个人知识库规模。向量库使用 Chroma，原因是本地持久化简单、Python 生态成熟、适合 MVP 快速验证，并且便于保存来源元数据。

11. Prompt 与引用机制

生成回答时，prompt 需要明确告诉模型：

- 只能基于提供的上下文回答。
- 如果上下文不足，要说明不确定。
- 回答后必须列出引用来源。
- 不要编造未出现在来源中的事实。

每个 chunk 需要保留来源文件、标题或章节、片段文本、来源名称和 chunk id。GUI 展示时，来源引用不只是技术元数据，而是用户信任系统的关键界面元素。

12. 当前验证结果

指标	当前结果
已索引文件数	163
Obsidian Markdown 文件	162
总 chunks	390
Obsidian chunks	385
本地文档 smoke-test chunks	5
使用入口	FastAPI Web GUI

13. 评估方法

评估不是只看模型回答是否流畅，而是看它能否完成个人知识库任务。

维度	判断方式
召回准确性	是否找到正确来源
Agent 任务判断	是否能区分检索、问答、索引维护和澄清场景
来源可追溯	是否展示可复核片段
回答可靠性	是否基于来源回答
使用效率	是否比手动搜索更快
本地可用性	是否能在当前电脑稳定运行

评估对比

测试问题样例

问题	期望来源	当前判断
Codex 每次打开应该读取哪些协作规则?	AI 协作规则	命中
本地知识助手 GUI 平时怎么用?	日常使用说明	命中
文件改动后怎么办?	索引更新说明	命中
换电脑时需要迁移哪些内容?	迁移操作指引	命中
本地知识助手 Agentic RAG 项目怎么介绍?	项目简介、压缩版、问答	命中
公开作品集不能放哪些内容?	公开封装说明	命中
企业知识库 RAG 项目的验证问题有哪些?	企业知识库验证表	命中
长文档索引要注意什么?	系统设计、协作规则	基本命中
GUI 截图在哪里?	页面原型说明、资产索引	命中
本地部署为什么不用云端?	产品设计文档	命中

小样本验证结果

当前验证围绕真实个人知识库任务构造了 10 条代表性测试问题，覆盖协作规则、GUI 使用、索引维护、换电脑迁移、项目介绍、公开封装和跨项目召回。

说明：测试问题来自真实个人知识库使用场景，而非人工构造的标准问答集。

指标	当前结果	说明
测试问题数量	10 条	来自真实使用场景，不是随机问答。
来源召回结果	9 条命中，1 条基本命中	基本命中表示召回方向正确，但仍需要用户进一步筛选。
回答可用性	约 8/10	人工判断回答是否基于来源、是否能辅助复盘。
Agent 任务判断	可区分检索、问答、索引维护和澄清类任务	复杂任务仍需要后续扩充问题集。

分层评估框架

为了避免只用“回答看起来是否流畅”判断效果，本项目把评估拆成三层：

层级	评估重点	典型问题
检索层	Top-K 是否召回正确来源，是否能覆盖同义表达	相关文件有没有被找回来
生成层	回答是否基于来源，是否标明不确定信息	有没有脱离材料编造
Agent 层	是否能判断任务类型，并选择检索、问答、索引说明或澄清	该直接回答，还是先让用户补充条件

14. 方案对比与取舍

决策	收益	成本
本地部署	隐私可控、成本低	跨设备迁移复杂
小模型	低配置电脑可运行	回答能力有限
Chroma	MVP 快速落地	企业级扩展能力有限
先检索再回答	降低幻觉	流程比聊天更复杂
手动索引优先	可控、实现简单	自动化程度不足
GUI 优先	更像产品，可日常使用	前端维护成本增加

与传统方式对比

维度	传统方式	本地知识助手 Agentic RAG
找资料	关键词搜索、按目录翻文件	自然语言语义检索，覆盖同义表达和跨文件线索
找答案	用户打开多个文件后手动拼结论	系统先召回来源，再组织可复核答案
可信度	依赖用户记忆和人工判断	回答绑定来源片段，可回到原文确认
连续任务	每次都重新搜索、重新交代背景	Memory 保留最近上下文，Router 判断是否继续检索

本质提升是：从“找文件”升级为“找答案 + 找依据”。

15. 限制与风险

- 当前小模型适合辅助总结，不适合复杂推理。
- 大批量索引会占用 CPU，需要分批执行。
- 文件路径变化后需要重新确认数据源配置。
- 扫描 PDF、图片和复杂表格暂未完整覆盖。
- 检索结果仍需要用户回到来源复核。
- 小样本验证规模有限，尚未覆盖复杂长链任务。
- Router 在问题意图模糊、上下文不足时仍可能误判任务类型。
- 多工具协同能力仍需通过 Planner、状态管理和失败回滚进一步增强。

16. Roadmap

版本	目标	说明
v0	基础 RAG	完成文档索引、向量检索、基础问答
v1	来源引用	每次回答展示来源文件和片段
v2	GUI 工作台	支持搜索、问答、索引任务和状态面板
v3	增量索引	文件改动后自动识别并刷新
v4	rerank	对召回结果进行二次排序
v5	保存回 Obsidian	将高价值回答沉淀为新笔记
v6	小团队版本探索	增加项目空间、基础权限、评估面板和本地私有部署说明

17. 产品化与延展路径

当前版本定位仍是个人知识工作台 MVP，不把它包装成已经商业化的企业产品。但从产品化角度看，它具备向小团队知识助手延展的基础：

- **目标用户延展**：从个人知识工作者扩展到 3–10 人的小团队，例如咨询、设计、产品和运营团队。
- **部署形态延展**：优先考虑本地私有部署或团队内网部署，而不是一开始做公开 SaaS。
- **能力延展**：增加项目空间、成员权限、资料更新提醒、评估面板和常见问题沉淀。
- **商业路径设想**：可以从一次性私有部署模板、轻量订阅版或面向垂直团队的知识库方案切入，但前提是先验证团队场景中的资料更新、权限和复核成本。

这部分更多体现产品经理对“从个人 MVP 到可产品化方案”的判断，不作为当前项目已经商业化的证明。

---

18. 产品洞察

AI 产品的关键不是“能回答”，而是“能被信任”

在个人知识库场景里，用户不缺一个会说话的模型，缺的是能帮他找回真实资料、解释依据、减少误判的系统。

一句话用户价值：用户不再需要记住“文件在哪”，只需要表达“我想要什么”，系统就能找到依据并给出可验证答案。

Agent 的引入，本质上是把系统从“信息检索工具”升级为“任务执行入口”。

本地知识助手 的价值在于控制权

云端 AI 工具很强，但不是所有资料都适合上传。本地知识助手 在模型能力上可能弱一些，但在隐私、可控和长期维护上更符合个人知识库场景。

GUI 是从 demo 到产品的分界线

同样的 RAG 能力，如果只能在终端使用，更像开发工具；如果有清楚的搜索、问答、索引和状态界面，用户才会把它当成日常产品。

评估要围绕任务，而不是围绕模型

这个项目的评估重点不是回答写得多漂亮，而是是否找到正确来源、是否能让用户复核、是否减少查找成本、是否能在真实电脑上稳定运行。

商业化思考不能脱离当前阶段

当前阶段更适合讲“产品化路径”和“可延展场景”，而不是讲规模化商业模式。先把个人场景做扎实，再讨论小团队私有知识助手，逻辑更可信。